HƯỚNG DẪN DEMO VẤN ĐÁP THỰC HÀNH 2

MỤC TIÊU: 8/8 ĐIỂM

Môi trường: Windows CMD

Môn học: Lập trình hướng dịch vụ (LTHDV / MSA) • **Dự án:** Ecommerce Monolith
Kiến trúc: Node.js Express + Prisma ORM + PostgreSQL + Docker Compose
Chuẩn phiếu chấm: Bảng chấm thực hành 2 (8 mục / 8.0 Điểm tối đa)

[KỊCH BẢN THỰC HIỆN TUẦN TỰ 8 CÂU - CHUẨN XÁC VÀ GỌN GÀNG]

1. **00s-08s (Lý thuyết):** Chỉ `src/` (phân tầng routes, controllers, middlewares) → Chỉ GitHub (chặn `.env`) → Chỉ `schema.prisma` (7 model, `@@id([oid,pid])`, transaction Serializable). *(Đạt Câu 1, 2, 3)*
2. **08s-18s (Terminal CMD & PostgreSQL):** Chạy lần lượt `docker compose ps` và `curl http://localhost:3001/health` → Vào `psql` gõ `\dt` và chèn 2 SP X, Y. *(Đạt Câu 4, 5)*
3. **18s-24s (Postman):** Bấm Send `Register` (role: normal, score: 10) → Bấm Send `Login` (token) → Bấm Send `Order` (mua X: 2, Y: 5). *(Đạt Câu 6, 7)*
4. **24s-30s (Đối chiếu cuối):** Trong `psql` xem 4 bảng: Thấy X còn 48, Y còn 45, Order & Shipment đầy đủ! *(Đạt Câu 8 & Trọn vẹn 8/8 Điểm!)*

CHI TIẾT 8 CÂU HỎI THEO PHIẾU CHẤM (CÂU 1 • CÂU 2)

CÂU 1: Giải thích cấu trúc project đạt chuẩn Monolithic — 1.0 ĐIỂM

Màn hình: VS Code mở rộng cây thư mục `src/`.

- Cấu trúc phân tầng rõ ràng: `routes/` (định tuyến), `controllers/` (nghiệp vụ), `middlewares/` (xác thực token Bearer & phân quyền admin), `validators/` (kiểm tra Zod).
- Toàn bộ phân hệ (Auth, User, Role, Membership, Product, Order, Shipment) nằm chung trong **1 ứng dụng Express duy nhất** và chia sẻ **1 cơ sở dữ liệu PostgreSQL** duy nhất, gọi hàm nội bộ (in-process).

> **[Nói mẫu với Giảng viên]**
> *"Dạ thưa thầy, dự án xây dựng chuẩn mô hình Monolithic theo cấu trúc phân tầng rõ ràng: thư mục routes/ định tuyến API, controllers/ xử lý nghiệp vụ, middlewares/ đảm nhiệm xác thực bảo mật và validators/ kiểm soát dữ liệu. Toàn bộ các phân hệ Auth, User, Product, Order và Shipment gom chung trong một mã nguồn Express và chia sẻ chung một cơ sở dữ liệu PostgreSQL."*

CÂU 2: Giải thích .gitignore và .env trên GitHub — 1.0 ĐIỂM

Màn hình: Trình duyệt Web mở repository GitHub của dự án.

- Mở file `.gitignore` → Chỉ vào các dòng: `.env`, `.env.*` và ngoại lệ `!.env.example`.
- Quay lại danh sách file → Chứng minh có file `.env.example` nhưng **tuyệt đối không có file** `.env`.

Lệnh kiểm tra tại máy local qua CMD: `type .gitignore`

> **[Nói mẫu với Giảng viên]**
> *"Dạ thưa thầy, trên repository GitHub em đã cấu hình .gitignore chặn toàn bộ file .env chứa mật khẩu kết nối database PostgreSQL và JWT_SECRET. Em chỉ commit file .env.example làm mẫu cấu hình để bảo đảm an toàn bảo mật."*

CHI TIẾT CÁC CÂU HỎI TIẾP THEO (CÂU 3 • CÂU 4 • CÂU 5)

CÂU 3: Giải thích mã nguồn Prisma ORM — 1.0 ĐIỂM

Màn hình: VS Code mở file `prisma/schema.prisma` và đoạn transaction trong `src/controllers/order.controller.js` (hàm `createOrder`).

- Định nghĩa đủ **7 model quan hệ**: `Role`, `MemberShip`, `User`, `Product`, `Order`, `OrderDetail`, `Shipment`.
- Bảng trung gian `OrderDetail` dùng **khóa chính kép**: `@@id([oid, pid])`.
- Luồng đặt hàng dùng `prisma.$transaction(..., { isolationLevel: 'Serializable' })` để đảm bảo toàn vẹn ACID.

> **[Nói mẫu với Giảng viên]**
> *"Prisma ORM định nghĩa đầy đủ 7 bảng quan hệ chuẩn theo thiết kế. Bảng chi tiết đơn OrderDetail sử dụng khóa chính kết hợp @@id([oid, pid]). Khi tạo đơn hàng trong order.controller.js, hệ thống sử dụng prisma.$transaction ở mức cô lập Serializable để đảm bảo toàn vẹn dữ liệu và tránh tranh chấp kho."*

CÂU 4: CLI: Compose hệ thống, system health check và liệt kê container — 1.0 ĐIỂM

Lệnh 1 — Xem danh sách container:

```
docker compose ps
```

Lệnh 2 — Kiểm tra trạng thái sức khỏe (Healthcheck):

```
curl http://localhost:3001/health
```

Kết quả hiển thị: Container `ecommerce-api` và `ecommerce-postgres` đều `**Up (healthy)**`; JSON trả về `{"status":"healthy","database":"connected"}`.

💡 **Liên kết Bài 1 & 3:** Bài 1 chỉ có 1 service; Bài 2 Monolith có **2 container** (chung 1 DB); Sang Bài 3 Microservices sẽ có **13 container** (6 services, 5 DB riêng, Kafka, Redis).

> **[Nói mẫu với Giảng viên]**
> *"Lệnh docker compose ps hiển thị đầy đủ 2 container ecommerce-api và ecommerce-postgres đang chạy đạt trạng thái Healthy. Endpoint /health thực hiện truy vấn SELECT 1 thật tới PostgreSQL và trả về HTTP 200 database: connected."*

CÂU 5: PostgreSQL CLI: Kiểm tra các bảng dữ liệu & Tạo mới 2 SP X, Y — 1.0 ĐIỂM

Bước 1: Vào container PostgreSQL bằng CMD:

```
docker exec -it ecommerce-postgres psql -U ecommerce -d ecommerce
```

Bước 2: Kiểm tra 7 bảng: `\dt`

Bước 3: Tạo 2 sản phẩm X (SL 50, giá 100) và Y (SL 50, giá 200):

```
INSERT INTO "Product" (pname, price, quantity) VALUES ('X', 100, 50), ('Y', 200, 50);
```

Bước 4: Xem lại sản phẩm vừa tạo:

```
SELECT * FROM "Product";
```

(Thấy X và Y có SL ban đầu là 50. Để nguyên cửa sổ psql này để làm tiếp Câu 8).

💡 **Liên kết Bài 1 & 3:** Bài 1 lưu Product vào MongoDB; Bài 2 lưu trong bảng quan hệ PostgreSQL; Sang Bài 3 Product được tách thành service riêng với database độc lập.

> **[Nói mẫu với Giảng viên]**
> *"Em truy cập vào container PostgreSQL bằng psql, dùng lệnh \\dt kiểm tra đủ 7 bảng theo thiết kế và chạy lệnh INSERT tạo mới 2 sản phẩm X và Y với số lượng ban đầu mỗi loại là 50."*

CHI TIẾT CÁC CÂU HỎI TIẾP THEO (CÂU 6 • CÂU 7)

CÂU 6: POSTMAN: Đăng ký (Role normal, Score 10) → Đăng nhập → Token — 1.0 ĐIỂM

Bước 1 - Đăng ký tài khoản mới:

- **Method:** `POST` • **URL:** `http://localhost:3001/api/auth/register`
- **Body (raw JSON):** `{"username":"khach01","fullname":"Nguyen Van A","password":"MatKhauDemo123!"}`
- **Kết quả:** Trả về `201 Created` → Chỉ vào: `"role": {"rolename": "normal"}` và `"membership": {"score": 10}`.

Bước 2 - Đăng nhập tài khoản:

- **Method:** `POST` • **URL:** `http://localhost:3001/api/auth/login`
- **Body (raw JSON):** `{"username":"khach01","password":"MatKhauDemo123!"}`
- **Kết quả:** Trả về `accessToken` (JWT Bearer Token). Postman đã tự động lưu token này vào biến.

> **[Nói mẫu với Giảng viên]**
> *"Em gửi request đăng ký tài khoản mới; hệ thống tự động gán vai trò normal và hạng thành viên có score: 10. Khi đăng nhập bằng tài khoản vừa tạo, API xác thực mật khẩu qua bcrypt và trả về chuỗi JWT accessToken."*

CÂU 7: POSTMAN: Luồng Đặt hàng (X: 2, Y: 5) → Detail → Shipment — 1.0 ĐIỂM

Bước 1 - Đặt hàng không có Token (Chứng minh chặn bảo mật):

- **Method:** `POST` • **URL:** `http://localhost:3001/api/orders` (không Header Authorization).
- **Body:** `{"items":[{"pid":1,"qty":2},{"pid":2,"qty":5}]}` → **Kết quả:** Báo lỗi `401 Unauthorized`.

Bước 2 - Đặt hàng có Token (Mua X số lượng 2, Y số lượng 5):

- Thêm Header: `Authorization: Bearer <accessToken>` → Nhấn **Send**.
- **Kết quả:** Trả về mã `201 Created`:
  - Thời gian đặt hàng lấy thời gian hiện hành (`createat`).
  - Chi tiết đơn hàng: Mua 2 sản phẩm X và 5 sản phẩm Y.
  - Tự động chuyển sang shipment: Trả về mảng `shipments: [{"status":"PENDING"}]`.

Bước 3 - Xem chi tiết đơn hàng: `GET http://localhost:3001/api/orders/1` (kèm Token) → Hiển thị chi tiết đơn và tổng tiền.

> **[Nói mẫu với Giảng viên]**
> *"Khi đặt hàng không có token, API từ chối với mã 401 Unauthorized. Khi có token hợp lệ, hệ thống tiến hành trừ kho mua 2 sản phẩm X và 5 sản phẩm Y, lưu thời gian tạo đơn hiện hành và tự động khởi tạo bản ghi Shipment trạng thái PENDING."*

CÂU 8 • DỌN DẸP DỮ LIỆU • BẢNG TỔNG HỢP

CÂU 8: CLI: Kiểm tra dữ liệu 4 bảng trong PostgreSQL — 1.0 ĐIỂM

Trong cửa sổ `psql`, bạn gõ lần lượt 4 câu lệnh SELECT:

```
SELECT * FROM "Product";
SELECT * FROM "Order";
SELECT * FROM "OrderDetail";
SELECT * FROM "Shipment";
\q
```

Số liệu đối chiếu thực tế:

- **Product:** X giảm từ 50 xuống **48** (50 - 2); Y giảm từ 50 xuống **45** (50 - 5).
- **Order & OrderDetail:** Có bản ghi mới với `oid: 1` và 2 dòng chi tiết X (qty 2), Y (qty 5).
- **Shipment:** Có bản ghi vận chuyển tương ứng với `status: PENDING`.

💡 **Liên kết Bài 1 & 3:** Bài 2 dùng 1 ACID transaction duy nhất cho Order & Shipment. Sang Bài 3 Microservices, Order Service sẽ gửi event Kafka `order-created` sang Shipment Service để tạo bản ghi giao hàng bất đồng bộ.

> **[Nói mẫu với Giảng viên]**
> *"Em dùng CLI truy vấn trực tiếp 4 bảng trong PostgreSQL: Bảng Order và OrderDetail ghi nhận đúng thông tin đơn hàng, kho sản phẩm đã được trừ chính xác (X còn 48, Y còn 45), và bảng Shipment đã sinh bản ghi giao hàng tương ứng."*

[CƠ CHẾ DỌN DẸP / RESET DỮ LIỆU ĐỂ LUYỆN TẬP LẠI]
Để tránh lỗi trùng username khi đăng ký lại hoặc tồn kho bị trừ dồn, hãy chọn 1 trong các cách sau:

Cách 1 (Nhanh nhất): Nhấp đúp file `reset_test_data.bat` (hoặc gõ trong CMD: `reset_test_data`).

Cách 2 (Lệnh SQL trực tiếp):

```
docker exec -i ecommerce-postgres psql -U ecommerce -d ecommerce -c "DELETE FROM \"Shipment\"; DELETE FROM \"OrderDetail\"; DELETE FROM \"Order\"; DELETE FROM \"Product\" WHERE pname IN ('X', 'Y'); DELETE FROM \"User\" WHERE username IN ('khach01', 'demo_user');"
```

Cách 3 (Reset xuất xưởng): `docker compose down -v` rồi `docker compose up -d`

Bước	Lệnh thực hiện từng bước (Không viết tắt)	Hành động & Liên hệ Bài 1, 3
Câu 4	1. `docker compose ps` 2. `curl http://localhost:3001/health`	Container Healthy, DB connected *(Bài 3 có 13 container)*
Câu 5	1. `docker exec -it ecommerce-postgres psql -U ecommerce -d ecommerce` 2. `\dt` 3. `INSERT INTO "Product" (pname, price, quantity) VALUES ('X', 100, 50), ('Y', 200, 50);` 4. `SELECT * FROM "Product";`	Xem 7 bảng và tạo 2 SP X, Y *(Bài 1 dùng MongoDB)*
Câu 8	1. `SELECT * FROM "Product";` 2. `SELECT * FROM "Order";` 3. `SELECT * FROM "OrderDetail";` 4. `SELECT * FROM "Shipment";` 5. `\q`	Kho trừ: **X còn 48, Y còn 45**; Đơn & Giao hàng lưu đủ *(Bài 3 dùng Kafka)*
Reset	`reset_test_data.bat`	Dọn sạch dữ liệu test để luyện tập lại từ đầu